



Predicting Stock Market Participation in the United States

Project for Introduction to Machine Learning for Economics (CEADV2401E)

Written By S176067

Date of submission: 21/12/2025

Number of pages: 10

Number of characters (non including appendix): 22741/22750

Data: (i)

Understanding who participates in the stock market is a central question in household finance. Identifying the characteristics of stock owners helps determine the main drivers of wealth accumulation and can assist governments in designing incentives. In this project I used household micro data from the 2022 U.S. Survey of Consumer Finances (SCF). The dataset is built by the Federal Reserve and contains almost 23000 observations and 356 variables. I downloaded the CSV file posted in the official website of the FED (click [First link](#) to download the official dataset). Variable definitions are provided in the [Second link](#). From the full extract I constructed a binary dependent variable, STOCK_PARTICIPATION, equal to 1 if the household's equity holdings (EQUITY) are strictly positive and 0 otherwise. I then selected a set of 19 control variables summarising key dimensions of the household's profile, including basic demographics (e.g. age, sex, married), labour market status, education, housing debt, liquidity and owned vehicles (see Appendix for full description). The result is a cross sectional dataset with one observation per household, a binary stock market participation indicator and a compact set of socioeconomic and financial controls, which I use as input for the subsequent machine learning analysis. Note that the dataset is slightly imbalanced: 36.96% of individuals do not own stocks (63.04% do). I didn't explicitly correct for this imbalance, as it is relatively mild. All the models I am going to use are split into training set (70% of the data) and test set (30% of data); a possible concern of the split I used is overfitting, however, results suggest that this split performs very well. In the Appendix you can find more details about metrics of the models I used (e.g. Sensitivity, Specificity...) and model comparison.

Logistic regression: (ii)

Logistic regression is used to model probabilities. While LPMs are easy to understand and interpret, they can obtain negative values or values larger than 1 (statistically inconsistent). Logistic regression solves this issue by using a logistic function instead of a linear function and applying Maximum Likelihood. We can see that LF, NOOWN and CHECKING_log are not statistically significant, while Hdebt is statistically significant at the 10% level and NVEHIC is statistically significant at the 5% level. Results are consistent with what we would expect: education, marital status, housing wealth and other key variables increase the probability of owning stocks (positive sign), while liquidity constraints decrease the probability of owning stocks (negative sign).

Variable	Estimate	Std. Error	z value	Pr(> z)	Signif.
(Intercept)	-9,253797	0,674829	-13,713	<2e-16	***
AGE	-0,009874	0,00191	-5,17	2.34e-07	***
HHSEX	-0,460035	0,065454	-7,028	2.09e-12	***
EDCL	0,592167	0,024318	24,351	<2e-16	***
MARRIED	0,613743	0,113328	5,416	6.11e-08	***
RACECL4	-0,205756	0,019998	-10,289	<2e-16	***
LF	0,169469	0,119073	1,423	0.1547	
OCCAT1	-0,552393	0,048297	-11,437	<2e-16	***
INDCAT	0,232619	0,055574	4,186	2.84e-05	***
FAMSTRUCT	0,232589	0,039575	5,877	4.17e-09	***

Variable	Estimate	Std. Error	z value	Pr(> z)	Signif.
HOUSECL	2,592994	0,257923	10,053	<2e-16	***
HLIQ	-1,260405	0,284837	-4,425	9.64e-06	***
HDEBT	0,096871	0,058882	1,645	0.0999	.
OWN	0,465706	0,079837	5,833	5.44e-09	***
NOWN	-0,10527	0,083428	-1,262	0.2070	
NVEHIC	0,17274	0,083267	2,075	0.0380	*
LIQ_log	0,326933	0,015879	20,589	<2e-16	***
CHECKING_log	0,001004	0,009671	0,104	0.9173	
SAVING_log	0,02228	0,005711	3,901	9.57e-05	***
HOMEQ_log	0,279876	0,021414	13,07	<2e-16	***

However, running the adjusted GVIF reveals that multicollinearity is affecting our results; as you can see HOMEQ_log, HOUSECL, NOOWN and NVEHIC display very high values:

HOMEQ_log	HOUSECL	NOWN	NVEHIC	INDCAT	MARRIED	FAMSTRUCT	LF	OCCAT1
34.55	32.50	20.11	19.08	7.04	6.48	5.81	5.66	4.25
LIQ_log	AGE	HHSEX	CHECKING_log	OWN	SAVING_log	HDEBT	HLIQ	EDCL
2.63	2.12	1.77	1.65	1.57	1.35	1.19	1.17	1.15

I decided to correct this multicollinearity problem by removing the variables with the highest GVIF values, specifically HOUSECL and NVEHIC. Results of the second regression are shown below:

Variable	Estimate	Std. Error	z value	Pr(> z)	Signif.
(Intercept)	-4,091302	0,443422	-9,227	<2e-16	***
AGE	-0,008172	0,001903	-4,294	1,8E-05	***
HHSEX	-0,462245	0,065545	-7,052	1,8E-12	***
EDCL	0,627017	0,024024	26,099	<2e-16	***
MARRIED	0,534297	0,112196	4,762	1,9E-06	***
RACECL4	-0,201169	0,019858	-10,13	<2e-16	***
LF	0,201485	0,118428	1,701	0,08888	.
OCCAT1	-0,502407	0,047805	-10,51	<2e-16	***
INDCAT	0,197892	0,055272	3,58	0,00034	***

Variable	Estimate	Std. Error	z value	Pr(> z)	Signif.
FAMSTRUCT	0,214843	0,039373	5,457	4,9E-08	***
HLIQ	-1,42526	0,284936	-5,002	5,7E-07	***
HDEBT	0,043107	0,05789	0,745	0,45649	
OWN	0,395587	0,076902	5,144	2,7E-07	***
NOWN	0,06542	0,024685	2,65	0,00804	**
LIQ_log	0,355168	0,015572	22,808	<2e-16	***
CHECKING_log	0,000169	0,009574	0,018	0,98592	
SAVING_log	0,01813	0,005629	3,221	0,00128	**
HOMEQ_log	0,071468	0,00454	15,742	<2e-16	***

Multicollinearity is no longer an issue:

INDCAT	MARRIED	FAMSTRUCT	LF	OCCAT1	LIQ_log	AGE	NOWN	HHSEX
7.06	6.39	5.78	5.70	4.20	2.56	2.12	1.82	1.78
CHECKING_log	HOMEQ_log	OWN	SAVING_log	HDEBT	HLIQ	EDCL	RACECL4	
1.64	1.57	1.48	1.34	1.17	1.16	1.13	1.08	

Then I implemented a stepwise approach for variable selection in order to remove the least relevant variables. CHECKING_log, HDEBT are excluded. The signs of the coefficients remain consistent, and their magnitudes closely align with those presented in the table above. Accuracy and log-loss are very similar.

Variable	Estimate	Std. Error	z value	Pr(> z)	Signif.
(Intercept)	-4,052811	0,439918	-9,213	<2e-16	***
AGE	-0,008297	0,001894	-4,38	1,2E-05	***
HHSEX	-0,460585	0,065488	-7,033	2E-12	***
EDCL	0,628014	0,023945	26,227	<2e-16	***
MARRIED	0,525584	0,111552	4,712	2,5E-06	***
RACECL4	-0,201088	0,019851	-10,13	<2e-16	***
LF	0,206369	0,118225	1,746	0,08089	.
OCCAT1	-0,503624	0,047743	-10,549	<2e-16	***

Variable	Estimate	Std. Error	z value	Pr(> z)	Signif.
INDCAT	0,198088	0,055264	3,584	0,00034	***
FAMSTRUCT	0,212389	0,039209	5,417	6,1E-08	***
HLIQ	-1,404966	0,28346	-4,956	7,2E-07	***
OWN	0,402085	0,076365	5,265	1,4E-07	***
NOWN	0,065116	0,024669	2,64	0,0083	**
LIQ_log	0,353794	0,013245	26,711	<2e-16	***
SAVING_log	0,018426	0,005547	3,322	0,00089	***
HOMEQ_log	0,071853	0,004511	15,93	<2e-16	***

The prediction on the test set has an accuracy of 81,09% and a log-loss of 0,4145.

Logistic regression	Actual_no_stock	Actual_stock
Pred_no_stock	1768	514
Pred_stock	789	3821

Test accuracy 81,09%

Support Vector Machines: (iii)

I started by standardizing the variables, setting a subsample of 8000 observations for tuning (1/3 of all the original sample size) using a 5-fold CV and setting cost = c(0.01, 0.1, 1, 10, 100). After tuning for the linear model I obtained a best cost equal to 0,1 and an expected accuracy of 81,15%. In total we found 7010 (3507,3503) support vectors which is a bit more than half the training sample. This means that it is not easy to separate classes linearly since there are many observations that appear close to these vectors. Kernel method has very different results compared to the linear model. To obtain a precise estimate I started by using a subsample of 8000 observations (similarly to the linear model) in order to tune hyperparameters. I ran a coarse grid search over a wide range of values using cross validation (5-fold CV), which identified a promising region of the parameter space. We obtained 4036 vectors (1934,2102) which allows for a larger best cost of 100 and a gamma of 0,1. I tried, also, a different specification by using a 3-fold CV and more close range parameters and results remained unchanged. Predictions on the test set have an accuracy close to 96,7% with an optimal threshold set to 0.44. The RBF specification delivers a much cleaner separation and higher predictive performance, confirming that stock participation is better captured by a non linear decision boundary than by a linear hyperplane. Results are summarized below:

Linear SVM	Actual_no_stock	Actual_stock
Pred_no_stock	1789	531
Pred_stock	768	3804

RBF SVM 5-fold	Actual_no_stock	Actual_stock
Pred_no_stock	2411	86
Pred_stock	146	4249

RBF SVM 3-fold	Actual_no_stock	Actual_stock
Pred_no_stock	2407	74
Pred_stock	150	4261

Model	Best_cost	Best_gamma	CV_expected_accuracy	CV_expected_sensitivity
Linear SVM	0,1	-	81,15%	87,75%
RBF SVM (5-Fold CV)	100	0,1	96,63%	98,02%
RBF SVM (3-fold CV)	150	0,15	96,75%	98,29%

The results that I obtain are consistent to what we would expect since SVMs are supervised learning methods that separate data points in a feature space by finding a hyperplane that maximizes the margin between classes. For this reason using a linear hyperplane gets results closer to the Logistic regression.

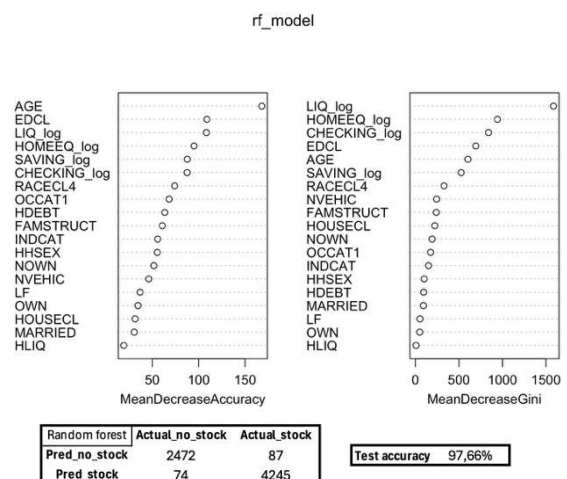
Classification trees: (iv)

I allowed the model to grow in a relatively flexible way by setting the control parameters to mincut = 3, minsize = 6 and mindev = 0.001, to explore a wide range of tree complexities. When moving to pruning, there is no difference in terms of deviance when selecting the size equal to 88 and 15 (see appendix [1](#)); we are going to select best_size = 15, as by using fewer nodes we get a more parsimonious and more interpretable tree (with less nodes there is more space for interpretability). The variables that are selected are the following: LIQ_log, EDCL, OCCAT1, NVEHIC, HDEBT and HOMEEQ_log with 15 terminal nodes. Our model has a test accuracy of 81,07%; unsurprisingly, it appears that the most important variable to predict whether an individual owns stock is liquidity with a threshold of 8,55 (roughly corresponding to 5.210 dollars) followed by home equity, education level number of vehicles and debt. The idea behind classification trees is to divide space into smaller areas selected by choosing thresholds on independent variables, so that observations within each region are as homogeneous as possible; the performance is very similar to linear SVM and the logistic regression.



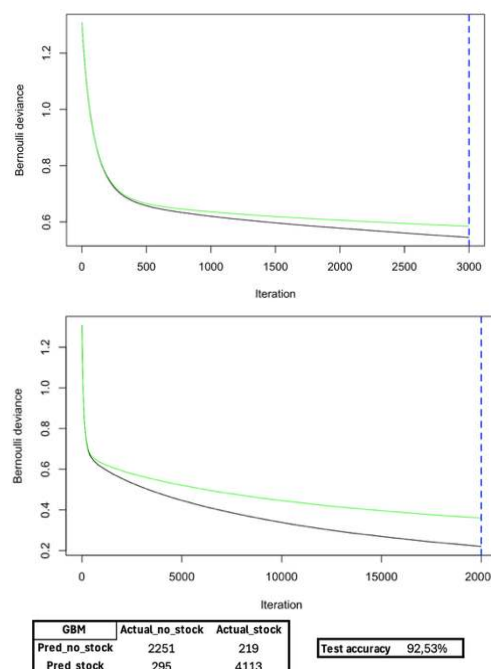
Random forest: (v)

I used 500 independent trees, each using a bootstrapped sample of the actual dataset; I set mtry=4 (at every split we are randomly selecting 4 variables out of the 19 available) using the rule of thumb that we saw in class $\sqrt{p}$. The out-of-bag error rate was 2,74%, confirming strong out of sample generalization. On the test set we achieved an accuracy of 97,66%, outperforming the logistic regression, the linear SVM model and the classification tree. As we know, random forest approach works by building decision trees on bootstrapped training; the performance of this model is probably due to the learning mechanism of Random Forests, which aggregates the predictions of many weakly correlated decision trees, reducing overfitting and capturing complex non-linear relationships. On the right corner we can see two plots; the first one shows the mean decrease accuracy that measures how relevant the variable is to predict who is a stockholder. The second plot is the mean decrease Gini and shows what are the variables that can create a more pure sample after cutting. These two tables have a very similar classification with the same variables at the top even with slightly different positions. The Random Forest assigns very high predicted probabilities to the correct class (close to 0 or 1), indicating strong model confidence. The significant improvement of RF compared to the classification trees suggests that stock ownership predictability is driven primarily by non linear interactions.



Boosted Trees (GBM): (vi)

Boosting improves model performance by building trees sequentially to correct the errors of previous iterations by fitting each new tree to the residuals: I started by setting the number of trees that the model uses to train at 3.000 and then tried several other specifications. We are going to use a Bernoulli distribution since the outcome is binary, cv folds are set to 5, interaction depth to 4 and finally shrinkage to 0,01. As we can see in the graph on the right setting the number of trees to 3.000 shows that the best number of trees we can choose is actually 3.000 (not very informative), based on Bernoulli deviance (loss function). For this reason I decided to try different specifications by increasing the number of trees to 5.000-8.000-12.000 and 20.000. Increasing the number of trees doesn't change the outcome; the number of trees selected by the model is always the maximum and the accuracy slightly increases; for this reason I will set the number of trees to 20.000. By doing so, I am able to increase the accuracy from 86,61% (3000 trees) to 95,77% (20000 trees) in the training set, while it achieved an impressive accuracy of 92.53% in the test set; It looks like the function is monotonically decreasing.



This model outperforms the simple classification tree; however, the random forest performs better than this model. The relevance of the variables agrees with classification tree and RF results (see Appendix I).

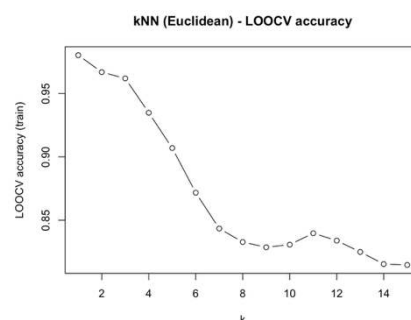
Classification tree		
	Actual_no_stock	Actual_stock
Pred_no_stock	1819	575
Pred_stock	727	3757
Accuracy:	81,07%	

Random forest		
	Actual_no_stock	Actual_stock
Pred_no_stock	2472	87
Pred_stock	74	4295
Accuracy:	97,66%	

GBM		
	Actual_no_stock	Actual_stock
Pred_no_stock	2251	219
Pred_stock	295	4113
Accuracy:	92,53%	

k-Nearest Neighbors (kNN): (vii)

I began by selecting the variables to avoid multicollinearity as I did with the logistic regression; then I proceeded by scaling the variables and by using euclidian distance. To select the best k we are going to use Leave One Out Cross Validation (LOOCV). The best k is selected to be =1; while this yields excellent predictive performance, it may reduce generalization by relying on near duplicate observations; for this reason I decided to set k=11; this means that the best way to predict the stock ownership in the test data is by computing the distance to the 11 closest points in the training data. As expected the accuracy in the training set has an accuracy of 85,41%; when we apply this model to the test set we get a very good performance since the accuracy is 83,58% using the Euclidean distance. Since finding the best k for both the Manhattan and Maximum distance is highly expensive in terms of computations we are going to use k=11 as well. The out of sample accuracy using the Manhattan distance is 83.82%, virtually identical to the Euclidean case. This indicates that nearest neighbour structure in the covariate space is highly stable. Very similar performance is achieved by the Maximum distance that has an accuracy of 82,50%. This confirms that KNN's performance is driven primarily by very local similarity patterns and is robust to alternative distance definitions. In conclusion, the KNN non-parametric approach that works by classifying data points based on the labels of their nearest neighbors, has a very similar performance to linear models.



kNN (Euclidean distance)			kNN (Manhattan distance)			kNN (Maximum / Chebyshev distance)		
	Actual_no_stock	Actual_stock		Actual_no_stock	Actual_stock		Actual_no_stock	Actual_stock
Pred_no_stock	1865	422	Pred_no_stock	1882	422	Pred_no_stock	1804	435
Pred_stock	710	3895	Pred_stock	693	3895	Pred_stock	771	3882
Accuracy	83,58%		Accuracy	83,82%		Accuracy	82,50%	

Results and conclusions:

These models confirm that owning stocks in the USA is something that can be predicted with high accuracy; non-linear models are better in predicting stock ownership since they are more flexible and since they can capture socio-economic characteristics better than linear models. Logit and classification trees provide us with some easy to interpret results, while RF, SVM and GBM provide us with more robust, accurate results. By looking at “Model metrics and performance comparison” in the appendix we can see that all the metrics are higher for these non-linear models; the second figure shows ROC curves; all our models perform better than the naïve estimator.

Note: one concern regarding the results I obtained is data leakage since accuracy is high in many models. I checked and removed some variables at the beginning so that there is no control variable directly related to stock ownership and I avoided multicollinearity problems by removing HOUSECL and NVEHIC in the logistic regression and in kNN estimation (as they are severely affected by it). The reason for this results might be endogeneity, however results seem to be robust.

Panel data in Machine Learning (viii):

Including a time dimension in a dataset (using panel data) comes with a set of problems such as serial correlation and non i.i.d. observations. This set of problems makes the random k-fold approach not feasible for panel datasets since the time dimension is relevant. We can solve this by using some validation strategies that take the time dimension into account by constructing a training set that is chronologically prior to the test set (or to use a rolling window approach respecting the temporal structure, Huber⁴).

Part of the variation could be driven by characteristics that are constant over time (in the stock analysis above, one characteristic could be risk aversion); to capture these effects we can use control for individual fixed effects to get a precise estimate using one-hot encoding (Chernozhukov⁵).

Logit model could be improved by stacking the panel in long format and include individual/time dummies and lagged variables (Varian, H.R.⁶). Trees models require more care, since many dummies can affect their performance; a better approach would be to use only lagged variables. For SVM a good approach could be to use kernel to capture non linear time dimensions (it can handle dummies better than tree models, but is computationally more costly). Finally, an approach that could be helpful to improve kNN could be to adapt panel data by defining a distance metric that also accounts for time and individual identity.

Semi-Supervised Learning (ix):

In semi supervised Learning, we have a small set of complete data (the value of the dependent variable and of the independent variables are known) and a large set of unknown outcomes. The idea is to frame the structure of the unlabeled data to improve the classifier (as this would lead to a better performance). The difference between the common SVM and the semi-supervised SVM approach lies in the fact that we don't know the classification of some of this individuals. Solution: the model tries to separate individuals by setting the hyperplane in low density areas by taking into consideration also unlabeled observations.

An example in which this approach could be potentially useful is classifying cancer types using data related to genetic expression, similarly to the example that we talked about in class related to the work of Guyon⁷. Semi supervised SVM could be a good approach to find results without necessarily knowing the outcome of every observation in the main dataset (notice that in her work she didn't use semi supervised because all the outcomes were known, however in principle this approach is feasible).

SHapley Additive exPlanations (SHAP) (x):

Shap values are used to explain individual (local) model predictions and/or global predictions. In particular they tell us how each feature has contributed, specifically how each feature has increased or decreased our prediction.

When applied to individual predictions, the goal is to understand how each characteristic contributed to explain the dependent variable of a single individual; this is done by computing the expected value of the outcome variable among all observations (all individuals) and by comparing it to the specific outcome of the individual of interest. Then, the machine looks at the specific individual independent variables and describes how much each of those variables contributed to the outcome. We can, also, derive global explanations by aggregating the local values. A common way of doing this is computing the mean of the absolute SHAP values across all observations. This tells us which features are the most important.

In their original work, Lundberg, S. M., & Lee describe the SHAP value as follows:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f_{S \cup \{i\}}(x_{S \cup \{i\}}) - f_S(x_S)] . \quad (4)$$

Where F is the total number of features, S is the number of features in S. In the second term, we are subtracting the predicted values using S from the predicted values of S plus feature i.

The problem of understanding feature contribution is framed by Efron as attribution, distinct from pure prediction or estimation. The main pitfalls of this approach are described by Molnar, C, where he says that if the features are correlated we can get biased estimates, that we shouldn't interpret them as causal explanations since they describe correlation and that it's sometimes difficult to interpret its results since all the variables are fit into the final model.

Unsupervised Learning:

i) The most flexible approach seen in class for handling the complex constraints typical of networks is mathematical optimization based clustering:

$$\text{minimize} \quad \sum_{l=1}^K \sum_{i=1}^n \sum_{j=1}^n \delta_{ij} y_{il} y_{jl}$$

the mathematical optimization approach offers a flexible framework to explicitly add these rules as constraints in the problem formulation.

Constraints can be incorporated as it follows:

$$\begin{aligned} \sum_{l=1}^K y_{il} &= 1, & i &= 1, \dots, n \\ y_{il} &\in \{0, 1\}, & i &= 1, \dots, n; \quad l = 1, \dots, K \end{aligned}$$

y_{il} is a binary decision variable that equals 1 if individual i is assigned to cluster l, and 0 otherwise.

We are basically measuring the dissimilarity between all possible pairs within the same cluster.

Must link constraints: These constraints force specific pairs of individuals to belong to the same cluster. ($y_{il} = y_{jl}$)

Cannot Link Constraints: These constraints ensure that specific pairs of individuals are separated into different clusters. ($y_{il} + y_{jl} \leq 1$)

Balance Constraints on Cardinality: These constraints control the desirable size of clusters. This is often referred to as imposing "cardinality constraints".

This approaches are very well described by Wagstaff in his papers.

ii) To define clusters we are going to minimise intra homogeneity and to maximise inter heterogeneity. The reference point is called prototype selected by minimizing the cost function (minimized through iterative procedures or mathematical optimization). So, in principle, we should be able to compare different models based on these 3 features; ideally we would like to choose the model with the highest inter heterogeneity, lowest intrahomogeneity and lowest global cost. However these measures are often difficult to compare directly as the number of clusters and the cost function can change based on the model we are using. For this reason is helpful to introduce other metrics: Here I included one type of metric for intra homogeneity, inter heterogeneity and global cost. Is useful to compare metrics related to these 3 measures because we can compare multiple characteristics of the cluster. They are described by Aggarwal & Reddy, and Hassan et al⁷. The major limit of WCSS is that it decreases by increasing the number of clusters, so unless the models we are comparing have the same number of clusters is unlikely to be a good metric to compare models. BCSS has a similar problem, since its value increases by increasing the number of clusters or when we deal with unbalanced data. Calinski-Harabasz index combines WCSS and BCSS in a ratio. It doesn't work very well with clusters that are compact or with different dimension.

1. Intra-Homogeneity (WCSS) – Minimize

Measures compactness (sum of squared distances to centroids).

$$W = \sum_{k=1}^K \sum_{i \in C_k} ||x_i - \mu_k||^2$$

Where K : total clusters, C_k : cluster k , x_i : data point, μ_k : centroid of cluster k .

2. Inter-Heterogeneity (BCSS) – Maximize

Measures separation (weighted sum of distances between centroids and global mean).

$$B = \sum_{k=1}^K n_k ||\mu_k - \mu||^2$$

Where n_k : points in cluster k , μ_k : centroid of cluster k , μ : global centroid.

3. Calinski-Harabasz Index (Global) – Maximize

Ratio of separation to compactness (validity score).

$$CH(K) = \frac{B/(K-1)}{W/(n-K)}$$

Where n : total observations, K : number of clusters, B, W : defined above.

Dimensionality Reduction:

i) PCA, often used with multivariate data, assumes that the data is zero centered (mean zero), and it works by searching the matrix of orthogonal vectors that minimizes the total sum squared of the Euclidean distances between points and their projection (or equivalently, by maximising the variance of the projected data). The resulting V space is a linear combination of these vectors (called principal components). MDS is a technique that places objects in a low dimensional space so that the distances between them resemble as closely as possible the original dissimilarities by squeezing the data into fewer dimensions while preserving how far apart points are. Independent Component Analysis (ICA)⁸ is a method used to unmix linearly mixed and statistic independent signals. The critical assumption of this model is that the data is a mixture of non gaussian and independent signals; by finding an appropriate linear transformation it is possible to recover the original signals without knowing the mixing process. The main advantage of this method is that is easier to interpret and that seeks for independency rather than uncorrelation, however, ICA is computationally heavier, sensitive to noise and requires independency/distribution assumptions. Note that ICA is not though to be a dimensionality reduction method: PCA and MDS reduce dimensionality by selecting only a few components, whereas standard ICA returns as many components as there are signals. When the sources are Gaussian, ICA collapses to PCA, and therefore its advantages are lost. See comparison in appendix¹.

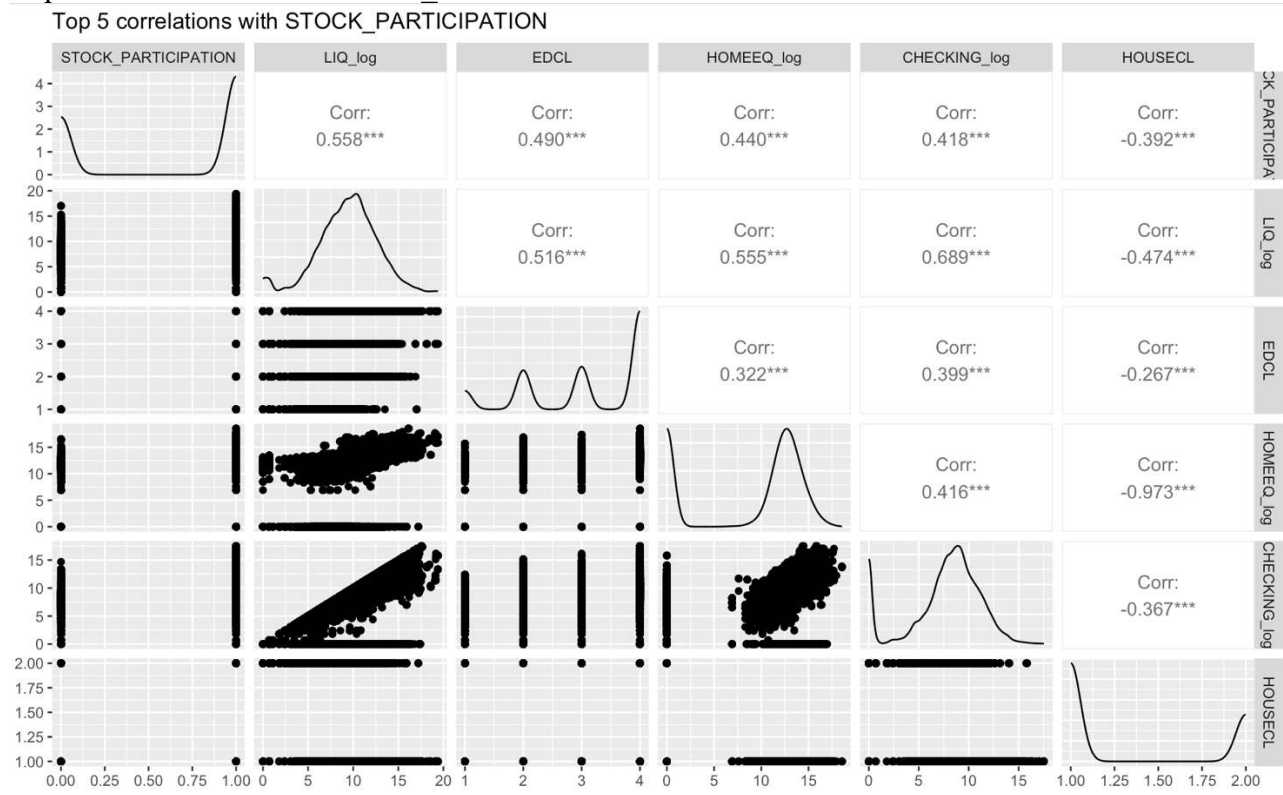
ii) To analyse dynamic data we can treat each time point as an observation in a latent space and then show how that point moves over time. Using PCA, we aggregate the observations in a score and we center it; The scores on the first PCs can be plotted in the plane or animated over time, highlighting trends and cycles; there also exist dynamic PCA variants that directly incorporate autocorrelation to extract components representing dominant dynamic modes⁹. With MDS, we construct the dissimilarity matrix between observations at different time points. We can build a distance matrix for each t , apply MDS to obtain a low dimensional configuration, and then align the configurations over time, producing animated maps in which the formation or splitting of clusters and changes in the similarity structure can be observed¹⁰.

Literature:

- ⁰Huber, M (2023), Causal analysis: Impact evaluation and Causal Machine Learning with applications in R
- ¹Chernozhukov, V. et al. (2025), Applied Causal Inference Powered by ML and AI
- ²Varian, H. R. (2014), Big data: New tricks for econometrics
- ³Guyon et al. (2002), Gene selection for cancer classification using support vector machines
- ⁴Lundberg, S. M., & Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions.
- ⁵Efron, B. (2020), Prediction, estimation, and attribution
- ⁶Wagstaff et al., 2000, Clustering with Instance-Level Constraints
- ⁶Wagstaff et al., 2001, Constrained K-means Clustering with Background Knowledge
- ⁷Aggarwal & Reddym Data Clustering: Algorithms and Applications
- ⁷Hassan et al., From A-to-Z review of clustering validation indices' (Neurocomputing, 2024)
- ⁸Christensen, R. (2019). Principal Components, Classical Multidimensional Scaling, and Factor Analysis. In Advanced Linear Modeling, Springer Texts in Statistics.
- ⁸Comon, P. (1994). Independent component analysis, a new concept? Signal Processing
- ⁹Christensen, Advanced Linear Modeling, 2019; Jolliffe, Principal Component Analysis, 2002
- ¹⁰Borg & Groenen, Modern Multidimensional Scaling, 2005; Buja et al., JCGS, 2008

Appendix

Top 5 correlations with STOCK_PARTICIPATION:



Variables description:

Variable	Type	Key values / labels
STOCK_PARTICIPATION	binary	0 = no equity holdings; 1 = holds equity → constructed using equity
EQUITY (not included)	numeric	>0 indicates some equity holdings; =0 indicates none.
AGE	numeric	Indicates how many years the individual has
HHSEX	categorical	1 = Male; 2 = Female
EDCL	categorical	Ordered categories from less than high school to graduate degree. (see second link)
MARRIED	binary	1 = Married/partnered; 0 = Otherwise
RACECL4	categorical	1 = White; 2 = Black/African-American; 3 = Hispanic/Latino; 4 = Other (mixed)
LF	binary	1 = work; 2 = doesn't work
OCCAT1	categorical	Major occupation groups (e.g. student, self employed ... see second link).
INDCAT	categorical	Major industry groups (e.g. mining, construction, manufacturing...see second link).
FAMSTRUCT	categorical	Family structure (e.g. Unmarried/partnered with children, Unmarried/partnered ... see second link).
HOUSECL	binary	Homeownership class: 1=owns ranch/farm/mobile home/house/condo/coop/etc., 2=otherwise
HLIQ	binary	Have any types of transactions accounts 1 = Yes; 0 = No
HDEBT	binary	Have any debts: 1 = Yes; 0 = No
OWN	binary	Have an owned vehicle: 1=yes, 0=no
NOWN	numeric	Number of owned vehicles
NVEHIC	numeric	Total number of vehicles (owned and leased)
LIQ	numeric	All types of transactions accounts (liquid assets); LIQ=CHECKING+SAVING+MMA+CALL+PREPAID
CHECKING	numeric	checking accounts other than money market
SAVING	numeric	Total value of savings
HOMEEQ	numeric	Home equity equals home value less all home secured debt

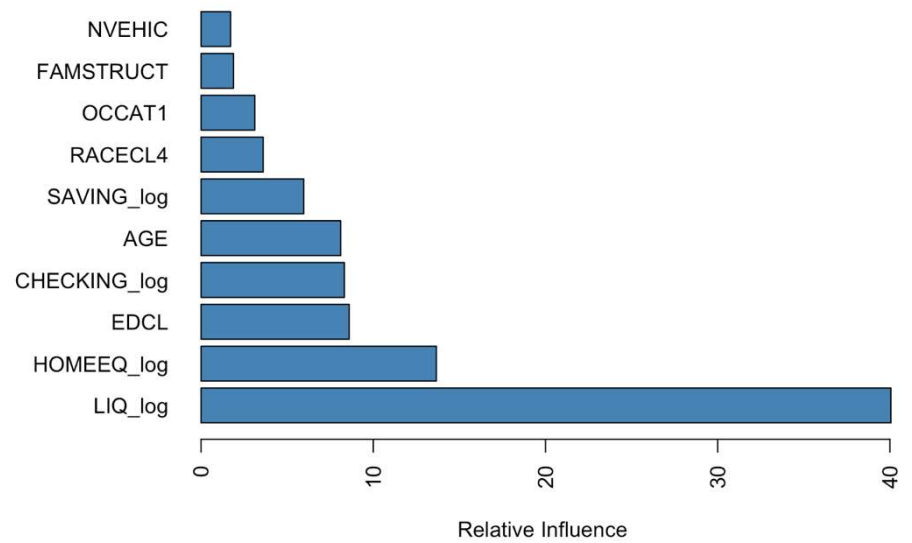
First link (full): <https://www.federalreserve.gov/econres/files/scfp2022excel.zip>

Second link (full): <https://www.federalreserve.gov/econres/files/bulletin.macro.txt>

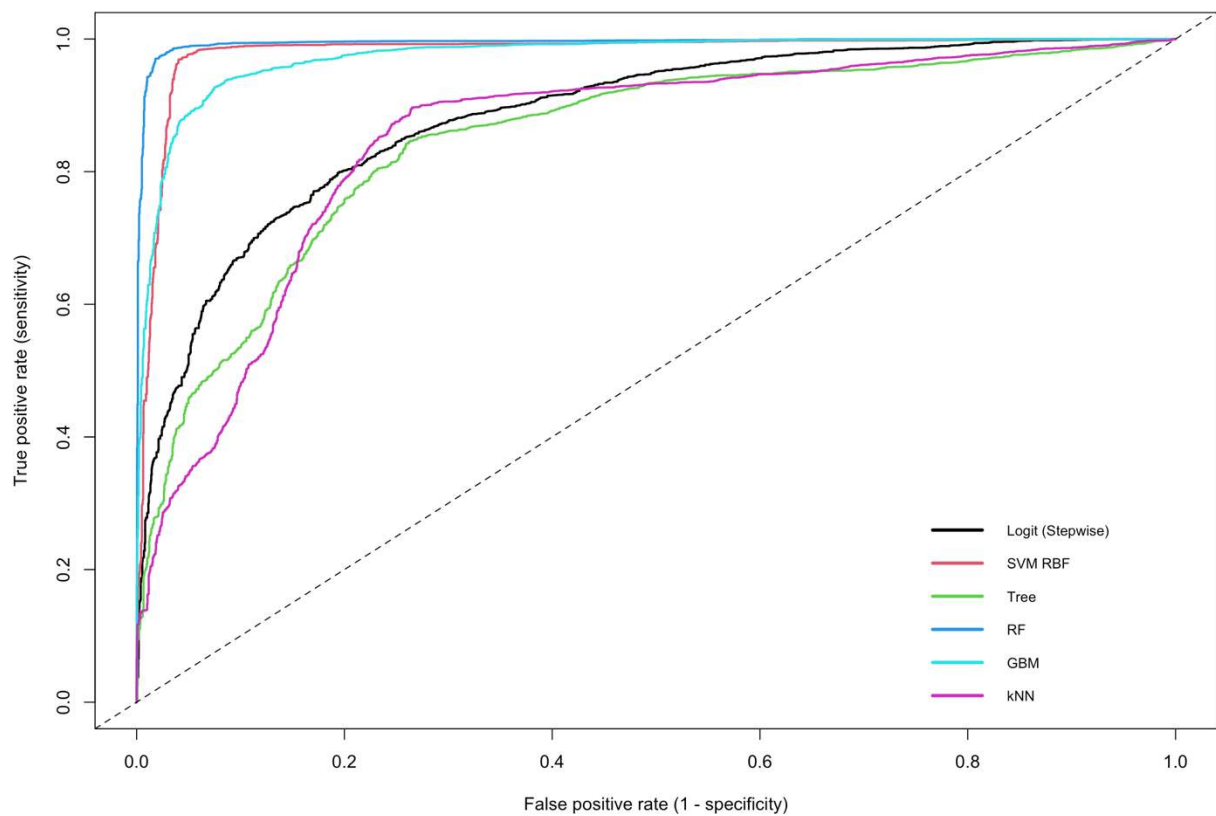
CT: Deviation by changing size⁵:

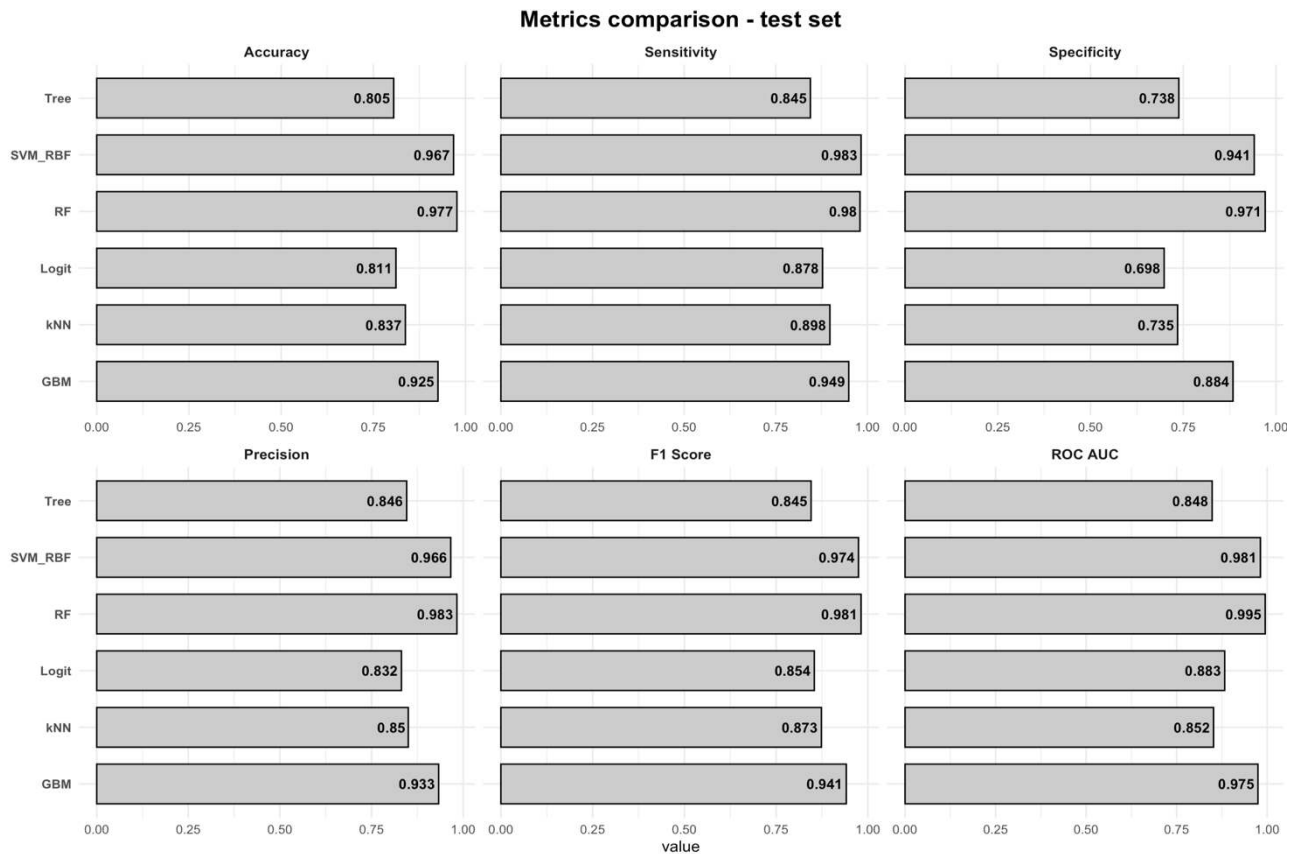
	Size	Deviance
1	88	3462
2	74	3462
3	71	3462
4	68	3462
5	65	3462
6	62	3462
7	54	3462
8	50	3462
9	45	3462
10	37	3462
11	36	3462
12	33	3462
13	32	3462
14	29	3462
15	27	3462
16	24	3462
17	22	3462
18	19	3462
19	17	3462
20	15	3462
21	9	3453
22	4	3453
23	2	3453
24	1	5919

GBM relevance of the variables⁵:



Model metrics and performance comparison⁵: *note that the best performing specifications are selected*
ROC curves by model (test)



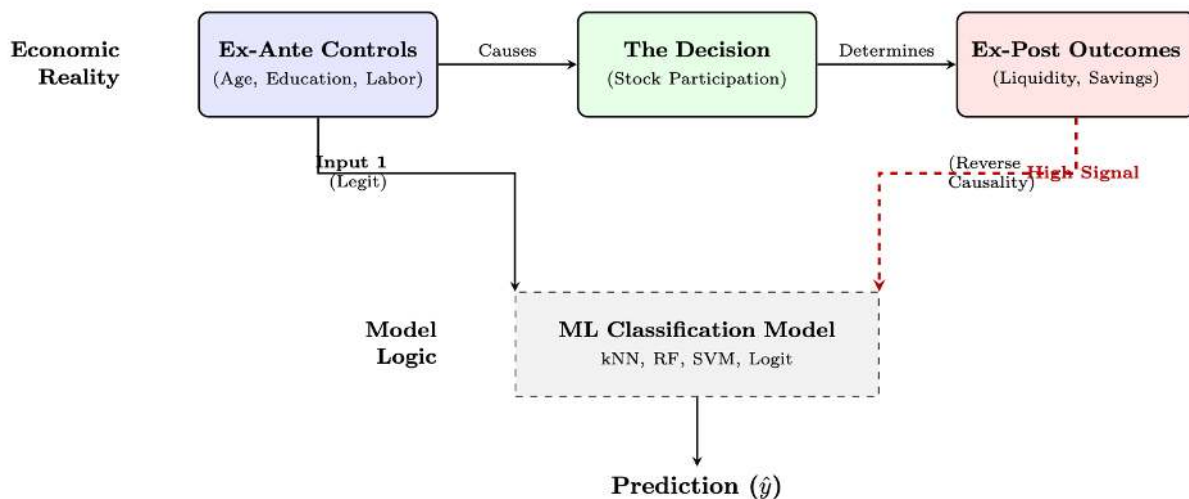


9 This table reports a stress test. When the y (stock ownership) is randomly shuffled, Random Forest accuracy drops to about 55%, close to the performance of the Naïve estimator. By running RF using different training sets and test sets the performance stays high suggesting that the model is correct.

Model	Accuracy	Sensitivity	Specificity	Precision	F1
Baseline majority	0,629	1	0	0,629	0,7722
RF shuffled Y	0,5532	0,7092	0,286	0,6299	0,6672
RF rep 1	0,9763	0,9835	0,9643	0,9787	0,9811
RF rep 2	0,9769	0,9848	0,9634	0,9787	0,9817
RF rep 3	0,9754	0,9776	0,9716	0,9837	0,9806
RF rep 4	0,9716	0,9806	0,9568	0,974	0,9773
RF rep 5	0,9721	0,9777	0,9624	0,9782	0,9779

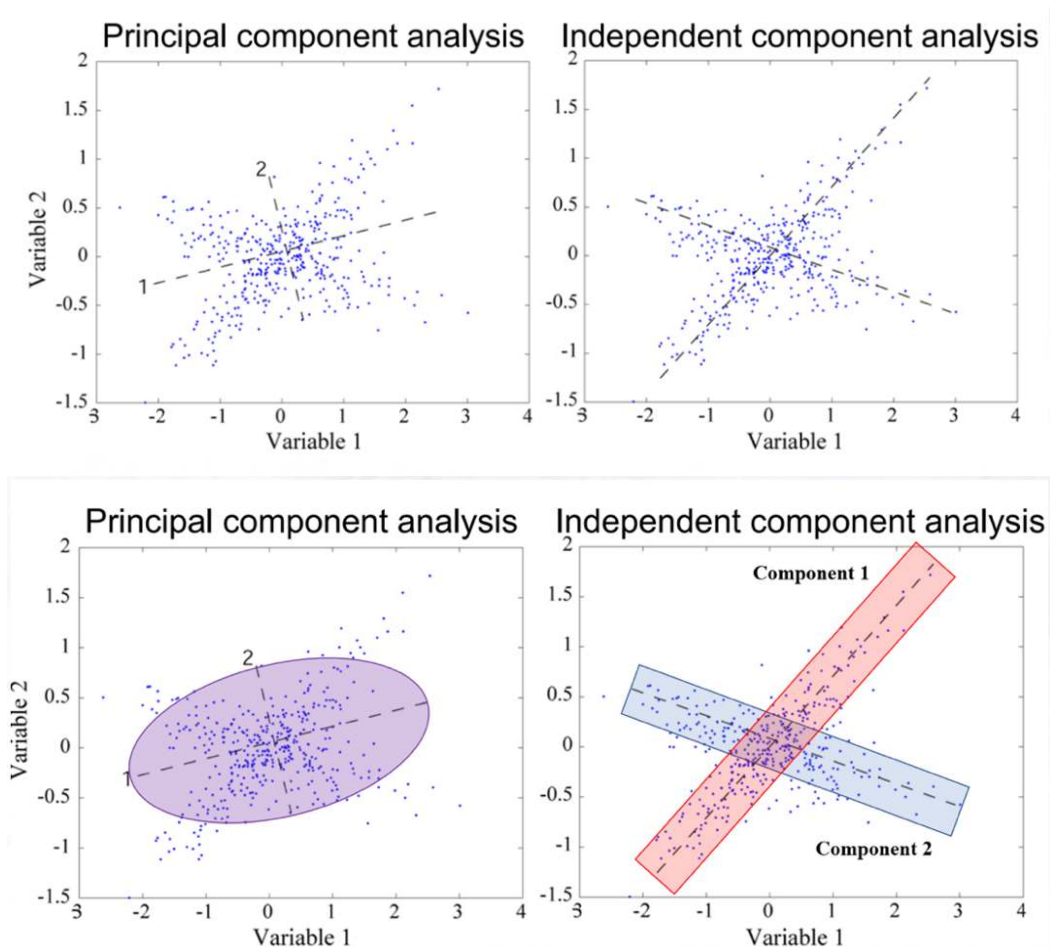
Same approach is used to evaluate SVM:

Model	Accuracy	Sensitivity	Specificity	Precision	F1
Baseline majority	0,629	1	0	0,629	0,7722
SVM RBF shuffled Y	0,5821	0,7566	0,2863	0,6425	0,6949
SVM RBF rep 1	0,9699652	0,9862	0,9419	0,967	0,9765
SVM RBF rep 2	0,9685142	0,9828	0,9441	0,9679	0,9752
SVM RBF rep 3	0,9693848	0,984	0,9441	0,9683	0,9761
SVM RBF rep 4	0,9676436	0,9786	0,9491	0,9703	0,9744
SVM RBF rep 5	0,9734475	0,9837	0,9557	0,9746	0,9791



The diagram illustrates the endogeneity problem. While socio-demographic factors (blue) influence the decision, financial outcomes (red) are determined by the decision. The ML model uses these downstream outcomes as inputs (red dashed line), creating a "reverse information flow" that boosts accuracy but prevents causal interpretation.

Comparison between PCA and ICA: (<https://loonylabs.org/2021/06/03/intro-to-ica/>)



CODE

```
rm(list = ls())

# --- Libraries ---
library(dplyr)
library(readxl)
library(tree)
library(randomForest)
library(gbm)
library(e1071)
library(FNN)
library(GGally)
library(car)
library(ggplot2)

set.seed(1)

file_path <- file.choose()
scf_raw <- read_excel(file_path)

# --- Helper Functions ---

compute_metrics <- function(cm, mod, lbl) {
  if (nrow(cm) < 2 || ncol(cm) < 2) return(NULL)

  cat(sprintf("\n--- %s : %s Metrics ---\n", mod, lbl))
  print(cm)

  TN <- cm[1, 1]
  FP <- cm[2, 1]
  FN <- cm[1, 2]
  TP <- cm[2, 2]

  acc <- (TP + TN) / sum(cm)
  sens <- TP / (TP + FN)
  spec <- TN / (TN + FP)
```

```

prec <- TP / (TP + FP)
f1 <- 2 * (prec * sens) / (prec + sens)

cat(sprintf("Accuracy: %.4f\n", acc))
cat(sprintf("Sensitivity: %.4f\n", sens))
cat(sprintf("Specificity: %.4f\n", spec))
cat(sprintf("Precision: %.4f\n", prec))
cat(sprintf("F1 Score: %.4f\n", f1))

invisible(list(Accuracy = acc, Sensitivity = sens,
               Specificity = spec, Precision = prec, F1 = f1))
}

minmax_norm <- function(tr, te) {
  mn <- min(tr, na.rm = TRUE)
  mx <- max(tr, na.rm = TRUE)
  den <- mx - mn

  if (den == 0) {
    return(list(train = tr * 0, test = te * 0))
  }

  list(train = (tr - mn) / den, test = (te - mn) / den)
}

# --- Data Preprocessing ---

scf_ls <- na.omit(scf_raw)

# Log Transformations
vars_log <- c("LIQ", "CHECKING", "SAVING")
scf_ls[paste0(vars_log, "_log")] <- log1p(scf_ls[vars_log])
scf_ls$HOMEEQ_log <- ifelse(scf_ls$HOMEEQ > 0, log1p(scf_ls$HOMEEQ), 0)

# Correlation Analysis
num_idx <- sapply(scf_ls, is.numeric)
cor_mat <- cor(scf_ls[, num_idx], use = "complete.obs")

```

```

top5 <- names(sort(abs(cor_mat[, "STOCK_PARTICIPATION"]),
                    decreasing = TRUE))[2:6]

# TryCatch per GGally
tryCatch({
  print(ggpairs(scf_ls[, c("STOCK_PARTICIPATION", top5)],
               title = "Top 5 correlations with STOCK_PARTICIPATION"))
}, error = function(e) NULL)

# Final Clean Dataset
scf_ls <- subset(scf_ls, select = -c(LIQ, CHECKING, SAVING, HOMEEQ))
scf_logit <- scf_ls
scf_svm <- scf_ls
scf_svm$STOCK_PARTICIPATION <- factor(scf_svm$STOCK_PARTICIPATION,
                                     levels = c(0, 1),
                                     labels = c("no_stock", "stock"))

# Train/Test Split
set.seed(1)
idx_ls <- sample(1:nrow(scf_ls), 0.7 * nrow(scf_ls))

logit_train <- scf_logit[idx_ls, ]
logit_test  <- scf_logit[-idx_ls, ]
train_split <- scf_svm[idx_ls, ]
test_split  <- scf_svm[-idx_ls, ]

# --- Logistic Regression & GVIF ---

# Model 1: Full
logit_model <- glm(STOCK_PARTICIPATION ~ .,
                  data = logit_train,
                  family = binomial)
summary(logit_model)

# GVIF Check 1
get_gvif <- function(model) {

```

```

val <- vif(model)

if (is.matrix(val)) val[, "GVIF^(1/(2*df))"]^2 else val
}

# Fit temp model removing pred cols for VIF check
temp_model <- glm(STOCK_PARTICIPATION ~ .,
                  data = select(logit_train, -matches("pred")),
                  family = binomial)
print(round(sort(get_gvif(temp_model), decreasing = TRUE), 2))

# Model 2: Cleaned (Removing HOUSECL, NVEHIC)
to_excl <- c("HOUSECL", "NVEHIC", "pred_prob", "pred_class")
logit_train_cl <- logit_train[, !names(logit_train) %in% to_excl]
logit_test_cl <- logit_test[, !names(logit_test) %in% to_excl]

logit_model_cl <- glm(STOCK_PARTICIPATION ~ .,
                     data = logit_train_cl,
                     family = binomial)
summary(logit_model_cl)

# GVIF Check 2
print(round(sort(get_gvif(logit_model_cl), decreasing = TRUE), 2))

# Out-of-Sample Logit Clean
logit_test_cl$pred_prob <- predict(logit_model_cl,
                                   newdata = logit_test_cl,
                                   type = "response")
logit_test_cl$pred_class <- ifelse(logit_test_cl$pred_prob > 0.5, 1, 0)

cm_logit <- table(Predicted = factor(logit_test_cl$pred_class, levels = 0:1),
                  Actual = factor(logit_test_cl$STOCK_PARTICIPATION,
                                   levels = 0:1))
compute_metrics(cm_logit, "Logit (Cleaned)", "Out-of-Sample")

logloss <- -mean(logit_test_cl$STOCK_PARTICIPATION * log(logit_test_cl$pred_prob) +
                 (1 - logit_test_cl$STOCK_PARTICIPATION) * log(1 - logit_test_cl$pred_prob))

```

```

cat("Log Loss (Out-of-Sample):", round(logloss, 4), "\n")

# Stepwise Selection
logit_step <- step(logit_model_cl, direction = "both", trace = 0)
summary(logit_step)
cat("Refined Model AIC:", round(logit_model_cl$aic, 2), "\n")
cat("Stepwise Model AIC:", round(logit_step$aic, 2), "\n")

# --- SVM Models ---

set.seed(1)
tune_sample <- train_split[sample(1:nrow(train_split), 8000), ]

# 1. Linear SVM
set.seed(1)
tune_lin <- tune.svm(STOCK_PARTICIPATION ~ .,
                    data = tune_sample,
                    kernel = "linear",
                    cost = c(0.01, 0.1, 1, 10, 100),
                    tunecontrol = tune.control(cross = 5))

final_lin <- svm(STOCK_PARTICIPATION ~ .,
                data = train_split,
                kernel = "linear",
                cost = tune_lin$best.parameters$cost,
                probability = TRUE)

summary(final_lin)
tab_lin_out <- table(Predicted = predict(final_lin, test_split),
                    Actual = test_split$STOCK_PARTICIPATION)
print(paste("Linear Test Accuracy:",
            round(sum(diag(tab_lin_out)) / sum(tab_lin_out) * 100, 2), "%"))

# 2. RBF SVM
set.seed(1)
tune_c <- tune.svm(STOCK_PARTICIPATION ~ .,

```

```

        data = tune_sample,
        kernel = "radial",
        cost = c(0.01, 0.1, 1, 10, 100),
        gamma = c(0.001, 0.01, 0.1),
        tunecontrol = tune.control(cross = 5))

cat("Coarse best cost:", tune_c$best.parameters$cost, "\n")
cat("Coarse best gamma:", tune_c$best.parameters$gamma, "\n")

set.seed(1)
tune_f <- tune.svm(STOCK_PARTICIPATION ~ .,
                  data = tune_sample,
                  kernel = "radial",
                  cost = tune_c$best.parameters$cost * c(0.5, 1, 1.5),
                  gamma = tune_c$best.parameters$gamma * c(0.5, 1, 1.5),
                  tunecontrol = tune.control(cross = 3))

bc <- tune_f$best.parameters$cost
bg <- tune_f$best.parameters$gamma
cat("Final RBF cost:", bc, " gamma:", bg, "\n")

set.seed(1)
final_rbf <- svm(STOCK_PARTICIPATION ~ .,
                 data = train_split,
                 kernel = "radial",
                 cost = bc,
                 gamma = bg,
                 probability = TRUE)

# 3. RBF Threshold Optimization
set.seed(1)
idx_val <- sample(nrow(train_split), min(8000, nrow(train_split)))
svm_sub <- svm(STOCK_PARTICIPATION ~ .,
               data = train_split[-idx_val, ],
               kernel = "radial",
               cost = bc,

```

```

        gamma = bg,
        probability = TRUE)

prob_val <- attr(predict(svm_sub, train_split[idx_val, ], probability = TRUE),
                  "probabilities")[, "stock"]

thresholds <- seq(0.3, 0.7, 0.01)
best_t <- thresholds[which.max(sapply(thresholds, function(t) {
  mean(ifelse(prob_val > t, "stock", "no_stock") ==
        train_split[idx_val, ]$STOCK_PARTICIPATION)
}))]
print(paste("Optimal Threshold:", best_t))

# Final RBF Metrics
prob_test <- attr(predict(final_rbf, test_split, probability = TRUE),
                  "probabilities")[, "stock"]
tab_rbf_def <- table(Predicted = predict(final_rbf, test_split),
                    Actual = test_split$STOCK_PARTICIPATION)

pred_opt <- ifelse(prob_test > best_t, "stock", "no_stock")
tab_rbf_opt <- table(Predicted = factor(pred_opt,
                                       levels = c("no_stock", "stock")),
                    Actual = test_split$STOCK_PARTICIPATION)

compute_metrics(tab_lin_out, "Linear SVM", "Out-of-Sample")
compute_metrics(tab_rbf_def, "RBF SVM (Default)", "Out-of-Sample")
compute_metrics(tab_rbf_opt, "RBF SVM (Optimized)", "Out-of-Sample")

# --- Tree-Based Methods ---

controls <- c("AGE", "HHSEX", "EDCL", "MARRIED", "RACECL4", "LF", "OCCAT1",
              "INDCAT", "FAMSTRUCT", "HOUSECL", "HLIQ", "HDEBT", "OWN", "NOWN",
              "NVEHIC", "LIQ", "CHECKING", "SAVING", "HOMEEQ")

scf_tree <- scf_raw
scf_tree$STOCK_CLASS <- factor(scf_tree$STOCK_PARTICIPATION, levels = 0:1,

```

```

labels = c("no_stock", "stock"))

treedata <- scf_tree[, c("STOCK_CLASS", controls)]

# Log Transforms
vars_tree <- c("LIQ", "CHECKING", "SAVING", "HOMEEQ")
treedata[paste0(vars_tree, "_log")] <- log1p(treedata[vars_tree])
treedata <- na.omit(treedata[, !names(treedata) %in% vars_tree])

set.seed(1)
train_id <- sample(1:nrow(treedata), floor(0.7 * nrow(treedata)))
train_set <- treedata[train_id, ]
test_set <- treedata[-train_id, ]

# 1. Tree
set.seed(1)
tree_full <- tree(STOCK_CLASS ~ ., data = train_set,
                  control = tree.control(nobs = nrow(train_set),
                                         mincut = 3, minsize = 6,
                                         mindev = 0.001))
cat("\nFull tree summary:\n"); print(summary(tree_full))

set.seed(2)
cv_tree <- cv.tree(tree_full, FUN = prune.misclass)
cat("\nOptimal Tree Size:", (best_size <- 15), "\n")

tree_pruned <- prune.misclass(tree_full, best = best_size)
compute_metrics(table(predict(tree_pruned, train_set, type = "class"),
                        train_set$STOCK_CLASS),
                "Single Tree", "In-Sample")
compute_metrics(table(predict(tree_pruned, test_set, type = "class"),
                        test_set$STOCK_CLASS),
                "Single Tree", "Out-of-Sample")

test_set$prob_tree_stock <- predict(tree_pruned, test_set,
                                   type = "vector")[, "stock"]

```

```

# 2. Random Forest

set.seed(1)

rf_model <- randomForest(STOCK_CLASS ~ ., data = train_set, ntree = 500,
                        mtry = floor(sqrt(ncol(train_set) - 1)),
                        importance = TRUE)

print(rf_model)

compute_metrics(table(predict(rf_model, train_set, type = "class"),
                            train_set$STOCK_CLASS),
                "Random Forest", "In-Sample")

compute_metrics(table(predict(rf_model, test_set, type = "class"),
                            test_set$STOCK_CLASS),
                "Random Forest", "Out-of-Sample")

test_set$prob_rf_stock <- predict(rf_model, test_set, type = "prob")[, "stock"]

# 3. GBM

train_gbm <- train_set
test_gbm <- test_set

train_gbm$STOCK_NUM <- ifelse(train_gbm$STOCK_CLASS == "stock", 1, 0)
train_gbm$STOCK_CLASS <- NULL
test_gbm$STOCK_NUM <- ifelse(test_gbm$STOCK_CLASS == "stock", 1, 0)
test_gbm$STOCK_CLASS <- NULL

set.seed(1)

gbm_model <- gbm(STOCK_NUM ~ ., data = train_gbm, distribution = "bernoulli",
                n.trees = 20000, interaction.depth = 4, shrinkage = 0.01,
                cv.folds = 5, verbose = FALSE)

best_iter <- gbm.perf(gbm_model, method = "cv", plot.it = FALSE)
cat("\nOptimal GBM Trees:", best_iter, "\n")

pred_gbm_in <- ifelse(predict(gbm_model, train_gbm, n.trees = best_iter,
                            type = "response") > 0.5, "stock", "no_stock")
compute_metrics(table(factor(pred_gbm_in, levels = c("no_stock", "stock")),
                            train_set$STOCK_CLASS),
                "GBM", "In-Sample")

```

```

test_set$prob_gbm_stock <- predict(gbm_model, test_gbm, n.trees = best_iter,
                                   type = "response")
pred_gbm_out <- ifelse(test_set$prob_gbm_stock > 0.5, "stock", "no_stock")
compute_metrics(table(factor(pred_gbm_out, levels = c("no_stock", "stock")),
                        test_set$STOCK_CLASS),
                "GBM", "Out-of-Sample")

# --- knn ---

scf_knn <- scf_raw
y_knn <- scf_knn$STOCK_PARTICIPATION
controls_knn <- c("AGE", "HHSEX", "EDCL", "MARRIED", "RACECL4", "LF", "OCCAT1",
                 "INDCAT", "FAMSTRUCT", "HLIQ", "HDEBT", "OWN", "NOWN",
                 "LIQ", "CHECKING", "SAVING", "HOMEEQ")

X_knn <- scf_knn[, controls_knn]
y_factor_knn <- factor(y_knn, levels = c(0, 1), labels = c("no_stock", "stock"))

set.seed(123)
n_knn <- nrow(scf_knn)
train_id_knn <- sample(seq_len(n_knn), size = floor(0.7 * n_knn))

X_train <- X_knn[train_id_knn, ]
y_train <- y_factor_knn[train_id_knn]
X_test  <- X_knn[-train_id_knn, ]
y_test  <- y_factor_knn[-train_id_knn]

# Normalization
X_train_norm <- X_train
X_test_norm  <- X_test

for (j in seq_along(controls_knn)) {
  if (is.numeric(X_train[[j]]) || is.integer(X_train[[j]])) {
    tmp <- minmax_norm(X_train[[j]], X_test[[j]])
    X_train_norm[[j]] <- tmp$train
  }
}

```

```

      X_test_norm[[j]] <- tmp$test
    }
  }

X_train_mat <- model.matrix(~ . - 1, data = X_train_norm)
X_test_mat  <- model.matrix(~ . - 1, data = X_test_norm)

# 1. Euclidean + LOOCV Tuning

best_k    <- NA
best_acc <- -Inf
acc_k     <- numeric(15)

set.seed(1)
for (k in 1:15) {
  cv_pred <- knn.cv(train = X_train_mat, cl = y_train, k = k)
  tab_cv  <- table(cv_pred, y_train)
  acc_k[k] <- sum(diag(tab_cv)) / sum(tab_cv)
}

best_k <- 11
print(data.frame(k = 1:15, LOOCV_Accuracy = round(acc_k, 4)))

plot(1:15, acc_k, type = "b",
     xlab = "k", ylab = "LOOCV accuracy (train)",
     main = "kNN (Euclidean) - LOOCV accuracy")

# In-Sample Euclidean
knn_eucl_in <- knn(train = X_train_mat, test = X_train_mat, cl = y_train, k = best_k)
conf_knn_eucl_in <- table(Predicted = knn_eucl_in, Actual = y_train)
compute_metrics(conf_knn_eucl_in, paste0("kNN Euclidean (k = ", best_k, ")"), "In-Sample")

# Out-of-Sample Euclidean
knn_eucl_out <- knn(train = X_train_mat, test = X_test_mat, cl = y_train, k = best_k)
conf_knn_eucl_out <- table(Predicted = knn_eucl_out, Actual = y_test)
compute_metrics(conf_knn_eucl_out, paste0("kNN Euclidean (k = ", best_k, ")"), "Out-of-Sample")

```

```

# 2. Manhattan (k = 11)
k_man <- best_k

preds_man <- apply(X_test_mat, 1, function(xi) {
  diffs <- sweep(X_train_mat, 2, xi, "-")
  d <- rowSums(abs(diffs))
  idx <- order(d)[1:k_man]
  names(which.max(table(y_train[idx])))
})

knn_man_out <- factor(preds_man, levels = levels(y_train))
conf_knn_man_out <- table(Predicted = knn_man_out, Actual = y_test)
compute_metrics(conf_knn_man_out, paste0("kNN Manhattan (k = ", k_man, ")"), "Out-of-Sample")

# 3. Maximum (k = 11)
k_max <- best_k

preds_max <- apply(X_test_mat, 1, function(xi) {
  diffs <- sweep(X_train_mat, 2, xi, "-")
  d <- apply(abs(diffs), 1, max)
  idx <- order(d)[1:k_max]
  names(which.max(table(y_train[idx])))
})

knn_max_out <- factor(preds_max, levels = levels(y_train))
conf_knn_max_out <- table(Predicted = knn_max_out, Actual = y_test)
compute_metrics(conf_knn_max_out, paste0("kNN Maximum (k = ", k_max, ")"), "Out-of-Sample")

```